Healthcare Appointment Booking System

Technical Architecture, Framework Guide & System Documentation

1. System Overview & Architecture

The Healthcare Appointment Booking System is a full-stack, enterprise-grade clinical scheduling application designed to manage doctor shift schedules, display real-time slot availability, prevent patient double-booking, and ensure consultations adhere strictly to doctor working hours. It consists of a **FastAPI** backend, a **SQLite** database tracked with **Alembic** migrations, a **ReactJS** single-page frontend with 12-hour AM/PM time formatting, and a generated **Python SDK**.

Component	Technology	Port / Location	Primary Role
Frontend Client	React 18 + Vite + Axios	http://localhost:3000	Clean single-page booking UI, 12h time slots, real-time sync
Backend API	FastAPI (Python 3.12)	http://localhost:8000	REST API, business logic, working-hour validation, CORS
Database	SQLite (SQLAlchemy ORM)	healthcare.db (Local)	Persistent relational storage for doctors and appointments
Migrations	Alembic	alembic/versions/	Versioned database schema migration management
Client SDK	OpenAPI Generator CLI	health_sdk/	Auto-generated typed Python SDK for API consumers
Live Broadcaster	WebSockets (/ws)	ws://localhost:8000/ws	Real-time live push updates when appointments are booked/cancelled

2. Deep Dive: How FastAPI Works

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. Here is what happens under the hood when a request arrives:

- **Asynchronous ASGI Server (Uvicorn):** FastAPI runs on an Asynchronous Server Gateway Interface (ASGI) engine. Requests are received non-blockingly, allowing high concurrency and low latency.
- **Automatic Type Validation (Pydantic v2):** When a JSON payload arrives, FastAPI uses Pydantic schemas to validate data types (e.g., verifying doctor_id is an integer, slot is a valid ISO timestamp, and start_hour < end_hour). If invalid, FastAPI automatically returns an HTTP 422/400 error without manual validation code.
- **Dependency Injection Pattern (Depends(get_db)):** Database connections are injected into endpoints per request. A database session is automatically opened for the endpoint and guaranteed to be cleanly closed upon completion.
- **Interactive OpenAPI & Swagger UI:** FastAPI automatically inspects function type signatures and generates an OpenAPI 3.1 specification (/openapi.json) and an interactive Swagger UI at **http://localhost:8000/docs**.

3. Where the Data is Stored (Database Architecture)

All application data is persistently stored in a local, single-file **SQLite** database located at: **healthcare.db** in your project root.

SQLite is a serverless, embedded relational database engine. Unlike MySQL or PostgreSQL, SQLite runs inside the FastAPI application process without needing a separate database daemon or server port.

Table Name	Columns & Types	Constraints & Description
doctors	id: INTEGER (PK) name: TEXT specialization: TEXT start_hour: INT end_hour: INT	Stores clinician profile and shift working hours (0-24). Auto-incrementing primary key.
appointments	id: INTEGER (PK) doctor_id: INT (FK) patient_name: TEXT slot: TIMESTAMP	Foreign key to doctors(id) with CASCADE delete. Stores patient appointment slot timestamps.
alembic_version	version_num: VARCHAR(32)	Maintained by Alembic to track which database migration revisions have been applied.

4. Backend Business Logic & Trick Logic (Hidden Complexity)

The coding challenge requires two core business rules (Trick Logic) to prevent scheduling conflicts:

Rule 1: Double-Booking Prevention

• **Mechanism:** When a booking request is made (POST /appointments/), the backend queries the database to see if an appointment already exists for the same doctor_id at the exact requested slot.
• **Outcome:** If an existing appointment is found, the server rejects the request with HTTP **409 Conflict**: *'Dr. [Name] already has an appointment booked at [Time]'*.
• **Isolation:** Double-booking checks are strictly doctor-specific. Two different doctors can both be booked at 10:00 AM simultaneously without conflict.

Rule 2: Working Hours Enforcement

• **Mechanism:** Each doctor has a defined shift: start_hour and end_hour (e.g., 9:00 AM to 5:00 PM).
• **Boundary Rule:** Appointments are 1-hour consultations. The appointment starting hour must satisfy: start_hour <= slot.hour < end_hour.
• **Example:** For a doctor working 9:00 AM to 5:00 PM (9 to 17), valid slots are 9:00 AM through 4:00 PM. Any request for 8:00 AM (before start) or 5:00 PM (end of shift) is rejected with HTTP **400 Bad Request**.

Rule 3: Slot Reutilization After Cancellation

• When an appointment is cancelled (DELETE /appointments/{id}), the slot is permanently deleted from the database. The slot immediately becomes available for other patients to book in real time.

5. Complete REST API Endpoints

Method & Route	Description	Key Parameters	Status
POST /doctors/	Add new doctor with shift hours	name, specialization, start_hour, end_hour	201 Created
GET /doctors/	List all registered physicians	None	200 OK
GET /doctors/{id}	Retrieve details of a specific doctor	id (path)	200 / 404
PUT /doctors/{id}	Edit/update doctor profile & hours	id (path), name, spec, hours	200 / 404
DELETE /doctors/{id}	Delete doctor & cascade appointments	id (path)	200 / 404
GET /doctors/{id}/availability	Get 1-hour slots & availability for date	id (path), query_date (YYYY-MM-DD)	200 OK
POST /appointments/	Book appointment (with trick logic)	doctor_id, patient_name, slot	201 / 400 / 409
GET /appointments/	List appointments with optional filters	doctor_id, patient_name (query)	200 OK
DELETE /appointments/{id}	Cancel appointment and free slot	id (path)	200 / 404
GET /stats	System dashboard & caseload analytics	None	200 OK
WS /ws	WebSocket connection for live updates	WebSocket handshake	101 Switch

6. ReactJS Frontend Implementation

The frontend client is built with React 18, Vite, and Axios for a lightweight, ultra-responsive user experience:

- **Single-Page Simplicity:** All functionality—viewing schedules, booking appointments, cancelling bookings, and managing doctors—is accessible on a single uncluttered dashboard without complicated tab switching or nested modals.
- **12-Hour AM/PM Time Formatting:** All doctor shifts, slot buttons, and scheduled timestamps are cleanly formatted in 12-hour AM/PM format (e.g., 9:00 AM – 5:00 PM, 10:00 AM, 2:00 PM).
- **Interactive Slot Picker:** Available consultation slots are displayed as green (Free) buttons. Clicking any free slot instantly selects the time and populates the booking box.
- **Doctor Management:** Provides an '+ Add New Doctor' button, an '═■ Edit' button to modify working hours or specialties, and a '■■ Delete' button to remove doctors.
- **Real-Time WebSocket Sync:** Listens to /ws. When an appointment is booked or cancelled in any window, all connected browser sessions automatically refresh the availability grid and schedule tables without reloading.

7. Platform Python SDK (OpenAPI Generator CLI)

Instead of writing manual API wrapper code, a production-grade Python SDK was compiled using the official OpenAPI Generator CLI tool: npm install -g @openapitools/openapi-generator-cli.

Exact Required SDK Usage:

```
from health_sdk.api.doctors_api import DoctorsApi
from health_sdk import ApiClient

client = ApiClient()
api = DoctorsApi(client)
doctors = api.list_doctors()
for d in doctors:
    print(f'Doctor: {d.name} ({d.specialization})')
```

8. Automated Testing & Verification

The test suite is powered by **Pytest** with 100% pass rate (39 passed tests) verifying:

- **Doctor Creation & Boundaries:** Verifies doctor registration, negative hours rejection, and start_hour >= end_hour constraints.
- **Double-Booking Trick Logic:** Verifies that booking Dr. A at 10:00 AM succeeds, and immediately re-booking Dr. A at 10:00 AM is rejected with HTTP 409 Conflict.
- **Working Hours Trick Logic:** Verifies that slots outside a doctor's shift are rejected with HTTP 400 Bad Request.
- **Cancellation & Slot Reutilization:** Verifies slot is freed and can be re-booked once cancelled.
- **Cascade Deletion:** Verifies deleting a doctor removes all associated bookings.

9. Execution Scripts & Quick Start

The project is 100% turnkey and can be launched on Windows using the included batch scripts:

- **setupdev.bat:** Creates Python venv, installs requirements, executes Alembic migrations, and runs frontend npm install.
- **runapplication.bat:** Starts FastAPI backend on port 8000 and React frontend on port 3000 concurrently.